

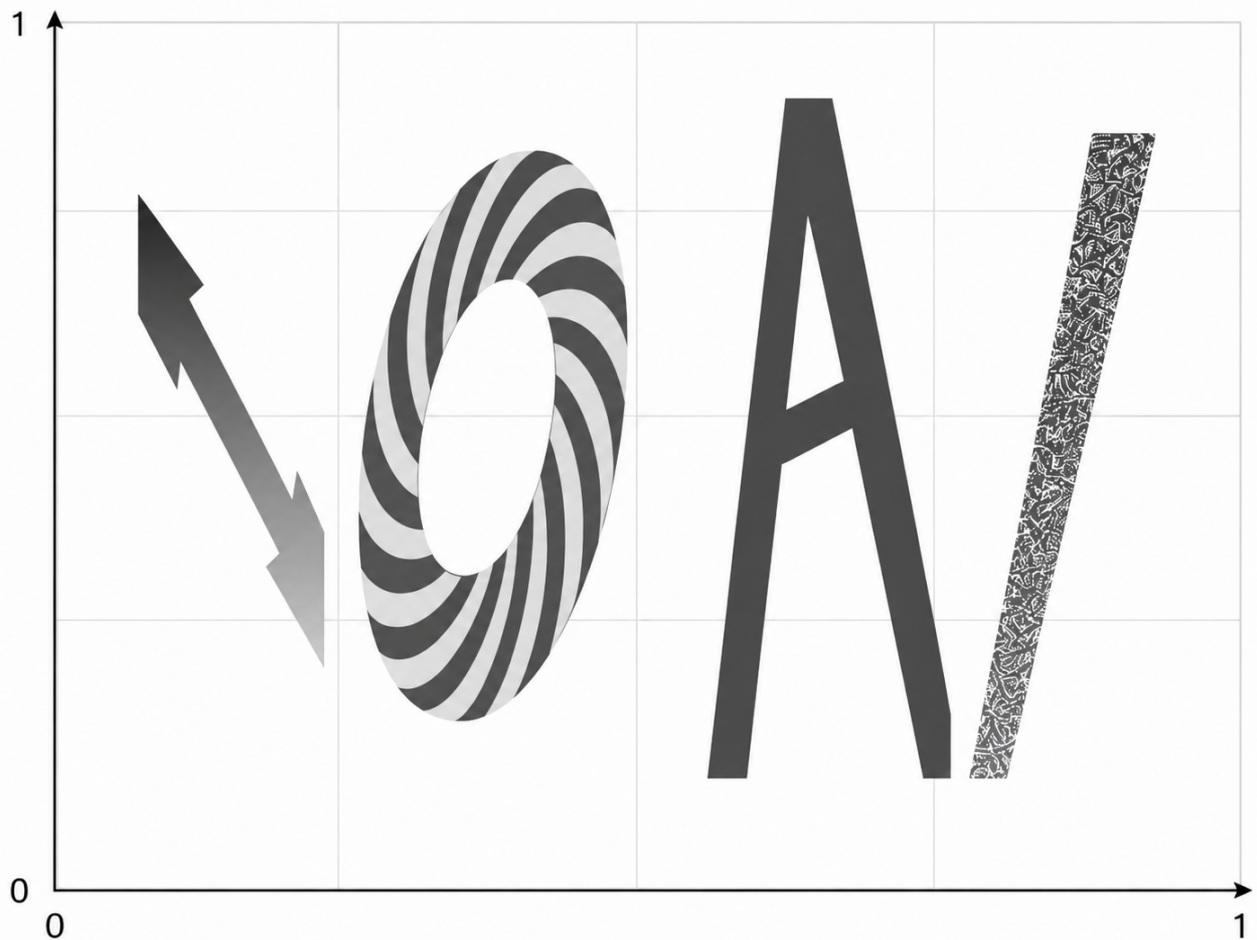
IOAI Field

- 시간: 5분
- 용량: 5 GB
- 솔루션 크기: `solution.ipynb`, `custom_model.py` 합쳐서 1 MB 이하
- 사전학습 모델: 없음, 처음부터 학습해야 하며, 채점 시 인터넷 사용 불가
- 베이스라인 점수: 31.2187

과제

아스타나 시장은 도시를 양식화된 IOAI 로고로 장식하고자 함. 그는 로고를 포함한 모든 것을 공간 함수 $F(x, y, \overline{W})$ 로 봄. 여기서 $x, y \in [0, 1]$ 는 2D 평면 상의 좌표를 나타냄, $\overline{W}$ 은 글자 색상과 각도 같은 양식적 속성을 정의하는 숨겨진 파라미터들의 집합임

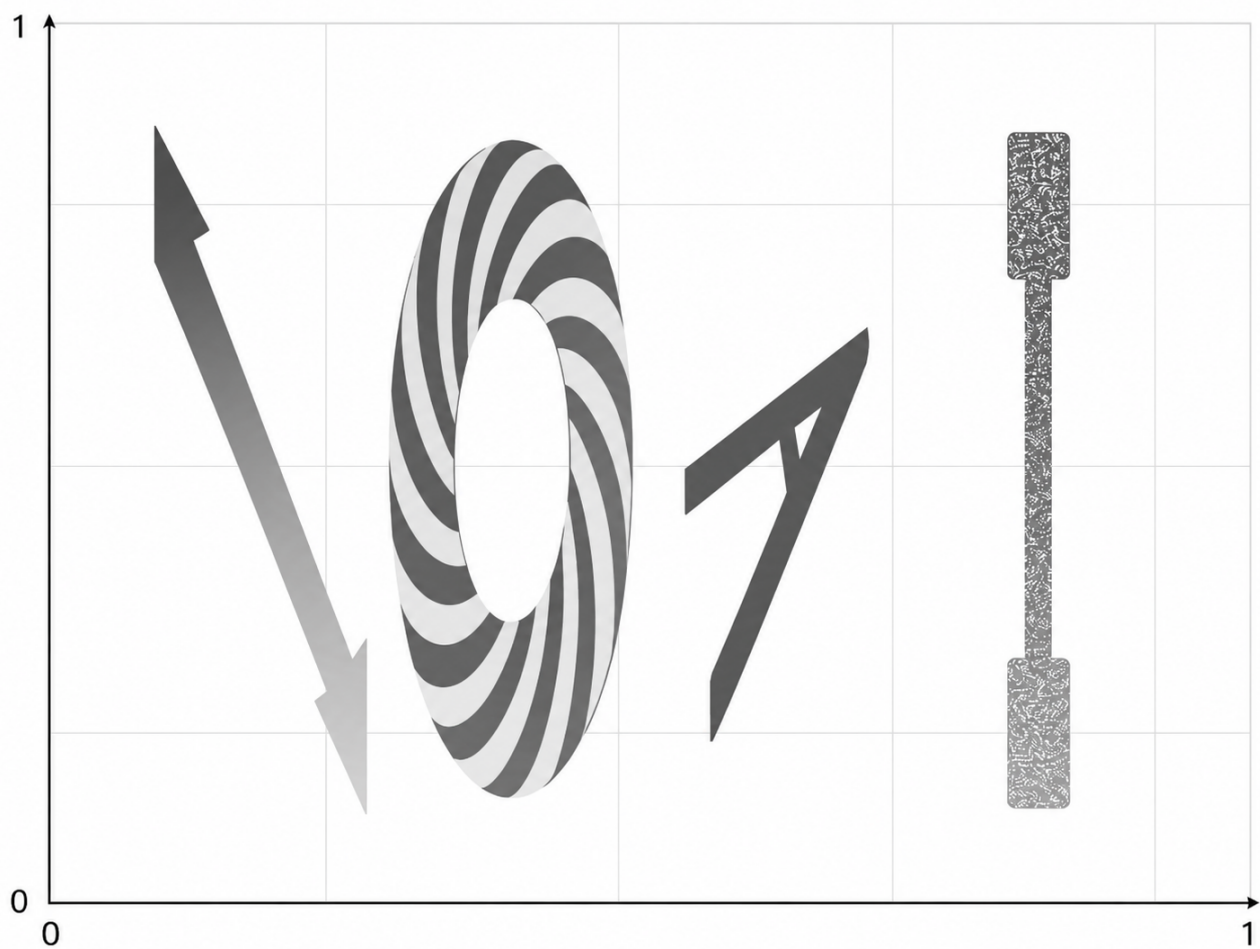
F 은 명시적인 방정식으로 표현하기에는 너무 복잡함, 과제는 이를 근사하는 신경망(neural network)을 학습시키는 것. 이 신경망은 임의의 좌표쌍 (x, y) 에 대해 **IOAI field** 값을 출력하여, 평면 전체에 걸친 로고의 완전한 히트맵(heatmap) 시각화를 생성. 다음은 특정한 숨겨진 파라미터 $\overline{W}$ 에 대한 F 의 히트맵 시각화 예시.

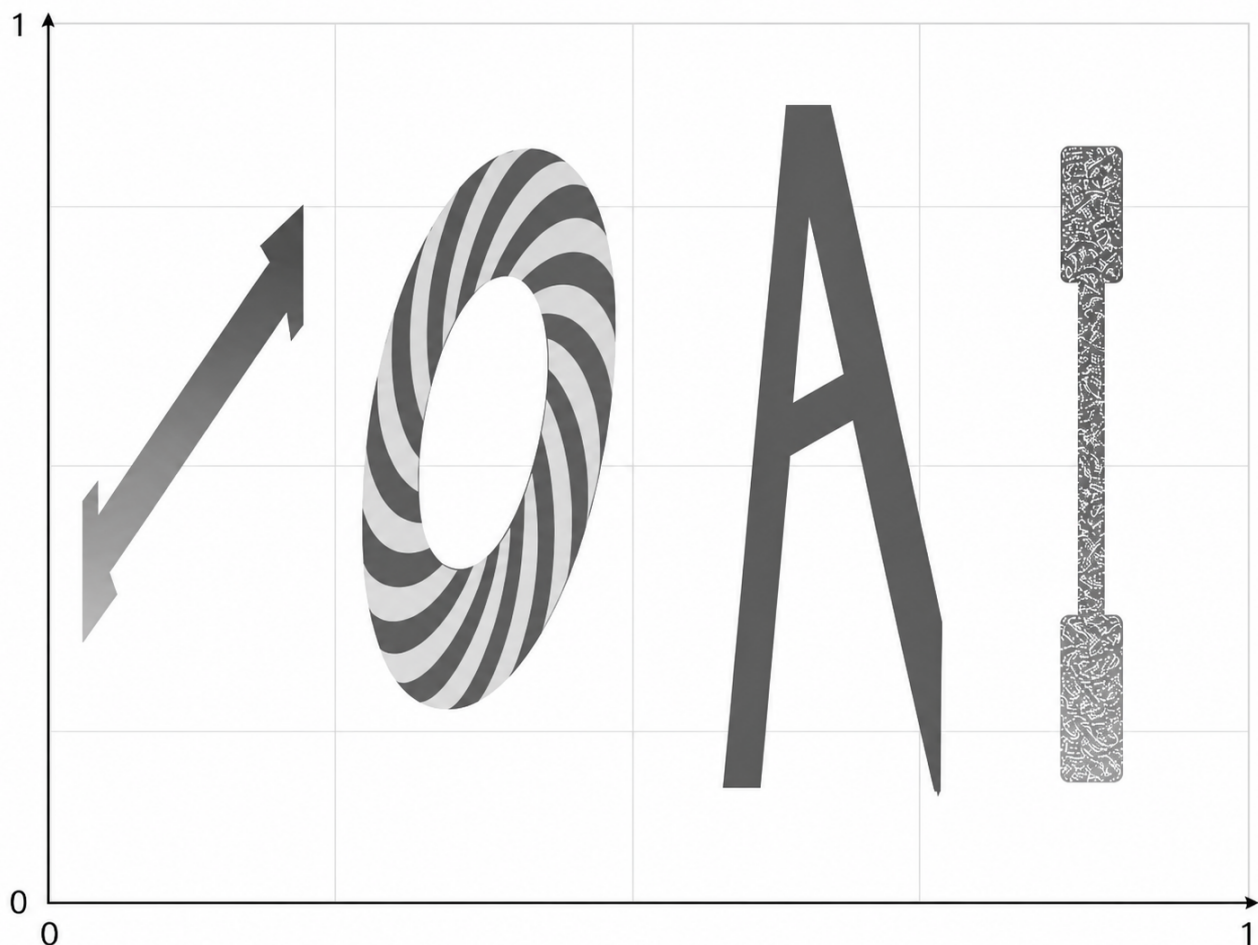


IOAI field는 무엇으로 구성되는가? 네 개의 글자와 배경

- 첫 번째 $\mathbb{I}$ 글자 내부의 값은 선형 그래디언트를 가지며 매우 큼($1e+10$ 이상)
- $\circ$ 글자 내부의 값은 나선형 패턴
- A 글자 내부의 값은 항상 -1
- 마지막 $\mathbb{I}$ 글자 내부의 값은 동일한 점에서 두 번 평가하더라도 $[-2026, 2026]$ 범위에서의 무작위 값이어야 함
- 글자 바깥에서의 값은 항상 0

이 함수는 숨겨진 파라미터 $\overline{W}$ 를 가짐 이는 글자의 스케일과 기울기, 그리고 첫 번째 $\mathbb{I}$ 글자 내부 값의 범위에 영향을 줌. 글자들은 서로 교차하지 않음. 다음은 서로 다른 $\overline{W}$ 에 대해 IOAI field가 어떻게 보이는지를 보여주는 예시임





제공되는 것:

이 문제에는 데이터셋이 제공되지 않음. 대신, `data/train_config/field_config.json`에 있는 JSON 설정 파일로 구성되는 생성기(generator) 함수가 제공됨

테스트 설정은 비공개이지만, 유사한 성격을 가짐. 과제는 주어진 생성기를 사용하여 원하는 만큼의 데이터로 학습하는 것. 여러분의 "train"과 "test" 분포는 동일한 생성기에서 생성되며, 다만 어떤 점 (x_i, y_i) 에서 평가될지는 알 수 없음.

제출물은 다음이어야함:

- `custom_model.py`으로 저장된 학습 모델 클래스. 이 모델은 `torch.nn.Module` 클래스를 상속해야 하며 `torch import`만 사용해야함. 또한 `solution.ipynb` 노트북에서 사용되는 `CustomModel` 클래스를 포함해야함.
- `model.pt` 가중치를 생성하는 `solution.ipynb` 노트북

채점

각 영역에 대해 최소 점수는 0이고 최대 점수는 1. 최종 점수는 다섯 개 영역(각 글자에 대한 네 개와 배경) 전체에 대해 평균을 낸 후 100을 곱한 값. **파라미터 페널티**가 있습니다:

모델의 파라미터 수가 20260개보다 많으면 점수가 절반으로 깎임

파라미터 수는 `sum(p.numel() for p in model.parameters())`으로 측정. 모델은 `PyTorch nn.Dropout`을 모델의 일부로 포함하여 확률적(stochastic) 모드로도 동작할 것으로 예상

표준 영역의 경우

각 영역 R (첫 번째 I 글자, O, A, Background)에 대해, 참값 v_i 과 예측값 $\hat{v}_i$ 을 갖는 $N_R = 512$ 개의 테스트 점 (x_i, y_i) 에서 모델을 평가. 주요 지표로는 정규화된 평균 절대 오차(MAE)를 사용함. MAE는 다음과 같이 정의됨:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

그리고 정규화는 다음과 같이 수행됨

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

여기서 $s_R > 0$ 은 스케일 상수임

마지막 I 글자 영역의 경우

이 영역에서는 **평가 시 dropout이 활성화됨**. 각 테스트 점 j 에 대해:

1. 모델을 $K = 10$ 번 실행하여 $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ 을 얻습니다.
2. 출력 중 하나라도 $[-2026, 2026]$ 범위를 벗어나면, $\text{pointScore}(j) = 0$ 입니다.
3. 그렇지 않으면, K 개 출력의 표준편차 σ_j 를 계산하여 점수로 변환합니다:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

여기서 $s_E > 0$ 은 고정된 스케일 상수임

영역 점수는 해당 영역의 모든 점에 대한 평균임

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

여기서 $N_E = K * N_R$ 입니다.

다양성 클수록 이 영역에서의 점수가 커짐. **PyTorch rand* 및 _uniform 함수를 포함하여 순수한 형태의 난수 사용 불가, 무작위성은 dropout이 활성화된 추론에서 비롯되어야함.**

제출 방법

1. `solution.ipynb`을 열고 모든 셀을 실행.
2. `custom_model.py`에서 `CustomModel` 모델을 개선.

3. 마지막 셀이 모델을 `model.pt` 파일로 저장하는지 확인.
4. JupyterLab Git 탭에서 `solution.ipynb`과 `custom_model.py`을 stage 하고, 커밋 메시지를 작성하여 커밋한 후 push.
5. Contest 페이지로 돌아가 **Submit**을 클릭합니다. 제출 코멘트는 이전 단계의 코멘트와 동일해야함.